

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342

Озернов Е.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Разработать и исследовать программу решения системы линейных уравнений методом Якоби (методом простых итераций) с выбором ведущего элемента, проверкой существования решения и невырожденности матрицы, а также провести тестирование на базовых, больших и специальных матрицах (в том числе матрицах Гильберта) с анализом корректности, времени и памяти.

Теоретический минимум

Метод Якоби (метод простых итераций для СЛУ) применяется для решения системы линейных уравнений вида $A * x = b$, где A — матрица коэффициентов размером $n \times n$, x — вектор неизвестных, b — вектор правых частей. Идея метода состоит в том, чтобы не решать систему напрямую, а строить последовательность приближений $x^{(0)}, x^{(1)}, x^{(2)}, \dots$, которая при выполнении условий сходимости стремится к точному решению.

Матрицу представляют в виде $A = D + L + U$, где D — диагональная часть, L — нижняя треугольная часть без диагонали, U — верхняя треугольная часть без диагонали. Тогда итерационная формула Якоби записывается как $x^{(k+1)} = D^{-1} * (b - (L + U) * x^{(k)})$. Покомпонентно: $x_i^{(k+1)} = (b_i - \sum_{j \neq i} (a_{ij} * x_j^{(k)})) / a_{ii}$. Ключевая особенность: на шаге $k+1$ используются только значения предыдущей итерации k .

Для корректной работы метода нужны ненулевые диагональные элементы a_{ii} , иначе формула деления не определена. Поэтому на практике применяют перестановки строк (выбор ведущего элемента), чтобы поставить на диагональ более крупные по модулю элементы и избежать нулей/почти нулей. Это не классический “прямой ход”, как в Гауссе, а предварительное преобразование системы для устойчивых итераций.

Сходимость метода определяется итерационной матрицей $B = -D^{-1}(L+U)$. Метод сходится тогда и только тогда, когда спектральный радиус $\rho(B) < 1$. На практике чаще используют достаточные условия: строгая диагональная преобладанность по строкам $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ гарантирует сходимость Якоби. Если это условие нарушено, метод может сходиться медленно или расходиться.

Проверка существования и числа решений системы выполняется через ранги. Если $\text{rank}(A) = \text{rank}([A|b])$, система совместна. Если при этом $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, решений бесконечно много. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система несовместна и решений нет. В вычислениях это проявляется, например, строкой вида $[0 \ 0 \ \dots \ 0 \ | \ c]$, где $c \neq 0$.

Критерий остановки итераций обычно задают по разности соседних приближений $\|x^{(k+1)} - x^{(k)}\|$ или по невязке $r^{(k)} = A \cdot x^{(k)} - b$ и норме $\|r^{(k)}\|$. В численных расчётах используется порог eps : значения с модулем меньше eps считаются нулевыми. Для контроля качества решения оценивают невязку $\|A \cdot x - b\|$, а если известно точное решение x^* , дополнительно ошибку $\|x - x^*\|$.

По трудоёмкости одна итерация метода Якоби для плотной матрицы требует порядка $O(n^2)$ операций, а для разреженной — $O(\text{nnz})$, где nnz — число ненулевых элементов. Общая стоимость составляет $O(\text{iterations} \cdot n^2)$ для плотного случая (или $O(\text{iterations} \cdot \text{nnz})$ для разреженного), поэтому метод особенно удобен для больших разреженных систем.

Выполнение работы

В ходе выполнения работы была реализована программа на языке C++, предназначенная для решения системы линейных уравнений методом Якоби (методом простых итераций) с поддержкой выбора ведущего элемента. В качестве способа выбора ведущего элемента использован частичный выбор по столбцу: перед началом итераций в каждом столбце среди доступных строк ищется элемент с максимальным модулем, после чего соответствующая строка при необходимости меняется местами с текущей. Такой подход позволяет снизить риск появления нулевых или слишком малых диагональных элементов и делает итерационный процесс более устойчивым.

На первом этапе работы программы выполняется проверка корректности входных данных. Контролируется, что матрица коэффициентов квадратная, а размер вектора правых частей совпадает с числом строк матрицы. После этого система приводится к внутреннему представлению в разреженном формате, что позволяет эффективно работать с матрицами больших размеров и не хранить лишние нулевые элементы.

Основная часть решения реализована через итерационную схему Якоби. Для каждой строки на очередной итерации вычисляется новое значение неизвестной по формуле $x_i^{(k+1)} = (b_i - \sum(a_{ij} * x_j^{(k)}, j \neq i)) / a_{ii}$, причём используются только значения предыдущего приближения. После обновления всех компонент считается норма разности соседних приближений $\|x^{(k+1)} - x^{(k)}\|_\infty$. Если эта величина становится меньше заданного порога точности, процесс останавливается и решение считается найденным. Также предусмотрено ограничение на максимальное число итераций, чтобы избежать бесконечного цикла при отсутствии сходимости.

В программе реализована проверка существования решения и контроль невырожденности матрицы в ходе вычислений. До запуска итераций выполняется анализ системы: быстрые проверки на противоречивость, а для

умеренных размеров дополнительно вычисляются ранги $\text{rank}(A)$ и $\text{rank}([A|b])$, по которым определяется тип системы. Если $\text{rank}(A) < \text{rank}([A|b])$, система несовместна и решений нет. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, система имеет бесконечно много решений. Если $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Дополнительно в процессе каждой итерации контролируются диагональные элементы: появление нулевого (или близкого к нулю) a_{ii} приводит к немедленному завершению с диагностическим сообщением, что фактически является проверкой вырожденности или некорректной перестановки для метода Якоби.

Для удобства использования решение разделено на несколько частей. В модуле решателя реализованы структуры данных, выбор ведущего элемента, проверки совместности/рангов, сам итерационный алгоритм и оценка памяти. В отдельном файле `main.cpp` организован ввод исходной системы, запуск решателя и вывод результата. Также создан файл `test.cpp`, содержащий расширенный набор тестов и автоматическую проверку корректности.

В процессе тестирования были рассмотрены разные типы входных данных. Сначала проверены базовые случаи с небольшими матрицами, где заранее известно корректное решение. Далее протестированы несовместные и вырожденные системы, в которых программа должна корректно определить отсутствие решения или бесконечное число решений. Эти проверки подтвердили правильность логики рангового анализа и классификации системы.

Отдельно проведены тесты на матрицах Гильберта как на специальных плохо обусловленных примерах. Для таких матриц метод Якоби обычно является неблагоприятным, поэтому основной целью было проверить корректную реакцию программы на трудные случаи: отсутствие сходимости, рост ошибок и корректное диагностическое завершение. Тесты подтвердили, что программа корректно обрабатывает такие сценарии и не выдаёт ложный «успех».

Кроме того, выполнены тесты на матрицах больших размеров. Помимо трёхдиагональных примеров 1000×1000 и 10000×10000 , добавлены более сложные структуры: циклические ленточные, пятидиагональные и разреженные

матрицы с дальними связями. Это позволило проверить работоспособность метода на больших не только «простых» матрицах и оценить поведение по времени и памяти в более реалистичных условиях.

В результате выполнения работы была создана программа, реализующая решение СЛУ методом Якоби с выбором ведущего элемента, проверкой существования решения, контролем невырожденности матрицы в процессе работы и расширенным тестированием на базовых, специальных и больших примерах. Полученная реализация соответствует поставленным требованиям и может применяться для учебных и прикладных задач при работе с разреженными системами большого размера.

Тестирование

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод

В ходе работы был реализован и протестирован метод Якоби (метод простых итераций) для решения СЛАУ с поддержкой выбора ведущего элемента (частичный выбор по столбцу). Программа корректно определяет тип решения (единственное, бесконечно много, решений нет), выполняет проверку совместности системы и контроль невырожденности матрицы в процессе вычислений через ранговый анализ, проверку специальных строк и контроль диагональных элементов на итерациях. Проведённое тестирование на базовых примерах, матрице Гильберта и больших размерностях (включая 1000×1000 , 10000×10000 и дополнительные крупные разреженные матрицы недиагональной структуры) подтвердило корректность реализации, ожидаемую численную устойчивость на подходящих классах матриц и работоспособность на больших входных данных. Дополнительно получены практические характеристики времени и памяти, что позволило оценить реальную вычислительную стоимость метода для крупных разреженных систем.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: jacobi_solver.h

```
#pragma once

#include <cstdlib>
#include <string>
#include <utility>
#include <vector>

// Перечисление стратегий выбора ведущего элемента.
// В текущей реализации реализованы:
// 1) NONE - перестановки строк не делаются.
// 2) PARTIAL_COLUMN_PIVOTING - для каждого столбца i выбирается строка с
//    максимальным по модулю элементом в этом столбце и переставляется
//    наверх.
enum class PivotStrategy {
    NONE = 0,
    PARTIAL_COLUMN_PIVOTING = 1
};

// Разреженная матрица в формате "список строк".
// Для каждой строки храним пары (столбец, значение) только для ненулевых
// коэффициентов. Это позволяет комфортно работать с размерами 1000 и
// 10000.
class SparseMatrix {
public:
    explicit SparseMatrix(std::size_t n = 0);

    std::size_t size() const;

    // Установка значения A[row][col].
    // Если |value| очень мало, элемент удаляется из разреженного
    хранения.
    void set(std::size_t row, std::size_t col, double value);

    // Получение значения A[row][col]. Если элемента в хранилище нет, это
    0.
    double get(std::size_t row, std::size_t col) const;

    // Доступ к списку ненулевых элементов конкретной строки.
    const std::vector<std::pair<std::size_t, double>>& row(std::size_t
row) const;

    // Перестановка двух строк (нужна для выбора ведущего элемента).
    void swapRows(std::size_t r1, std::size_t r2);

    // Оценка памяти, занимаемой матрицей, в байтах.
    std::size_t memoryBytes() const;

private:
    std::vector<std::vector<std::pair<std::size_t, double>>> rows_;
};

// Классификация системы по наличию решений.
enum class SolutionType {
    UNIQUE, // Единственное решение
```

```

    INFINITE,          // Бесконечно много решений
    NONE,              // Решений нет
    UNDETERMINED       // Однозначно определить по быстрой проверке не
удалось
};

// Параметры метода Якоби.
struct JacobiConfig {
    double tolerance = 1e-10;          // Критерий остановки по
норме ||x(k+1)-x(k)||inf
    std::size_t maxIterations = 100000; // Максимум итераций
    PivotStrategy pivotStrategy = PivotStrategy::PARTIAL_COLUMN_PIVOTING;
    double zeroEps = 1e-12;           // Порог "считаем нулём"
    std::size_t exactRankCheckLimit = 250; // До какого n делаем
точную ранговую проверку
};

// Результат работы решателя.
struct SolveResult {
    bool success = false;              // Удалось ли получить
решение
    bool converged = false;            // Сошёлся ли итерационный
процесс
    SolutionType solutionType = SolutionType::UNIQUE;
    std::size_t iterations = 0;        // Количество выполненных
итераций
    std::vector<double> x;              // Найденный вектор
решения (если есть)
    std::string message;               // Подробный статус
    double achievedDelta = 0.0;        // Фактическая ||x(k+1)-
x(k)||inf на последней итерации
    std::size_t estimatedMemoryBytes = 0; // Оценка использованной
памяти
};

// Основная функция решения СЛУ Ax=b методом Якоби.
SolveResult solveByJacobi(SparseMatrix a, std::vector<double> b, const
JacobiConfig& config);

// Вспомогательная функция для генерации матрицы Гильберта размера n.
SparseMatrix buildHilbertMatrix(std::size_t n);

// Вспомогательная функция для генерации трёхдиагональной матрицы:
// diagValue на главной диагонали, offDiagValue на соседних.
SparseMatrix buildTridiagonalMatrix(std::size_t n, double diagValue,
double offDiagValue);

```

Название файла: gauss_solver.cpp

```

#include "jacobi_solver.h"

#include <algorithm>
#include <cmath>
#include <limits>
#include <sstream>
#include <stdexcept>

```

```

namespace {

// Служебная структура с результатом рангового анализа.
struct RankAnalysisResult {
    SolutionType type = SolutionType::UNDETERMINED;
    std::string message;
};

// Проверка: есть ли в строке только нули.
bool isZeroRow(const std::vector<std::pair<std::size_t, double>>& row,
double eps) {
    for (const auto& item : row) {
        if (std::abs(item.second) > eps) {
            return false;
        }
    }
    return true;
}

// Быстрый контроль на противоречивость системы:
// если есть строка 0 ... 0 | bi, где bi != 0, решений точно нет.
bool hasInconsistentZeroRow(const SparseMatrix& a, const
std::vector<double>& b, double eps) {
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; ++i) {
        if (isZeroRow(a.row(i), eps) && std::abs(b[i]) > eps) {
            return true;
        }
    }
    return false;
}

// Проверка строгого диагонального преобладания по строкам:
// |aii| > sum{j!=i} |aij| для каждой строки i.
// Для таких матриц:
// 1) матрица невырождена,
// 2) метод Якоби гарантированно сходится.
bool isStrictlyDiagonallyDominant(const SparseMatrix& a, double eps) {
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; ++i) {
        double diagAbs = 0.0;
        double offDiagSum = 0.0;
        for (const auto& [col, value] : a.row(i)) {
            if (col == i) {
                diagAbs = std::abs(value);
            } else {
                offDiagSum += std::abs(value);
            }
        }
        if (!(diagAbs > offDiagSum + eps)) {
            return false;
        }
    }
    return true;
}

// Перестановка строк по стратегии частичного выбора главного элемента по
столбцу.

```

```

// Для каждого столбца i выбирается строка r >= i с максимальным |ari|.
void applyPartialColumnPivoting(SparseMatrix& a, std::vector<double>& b,
double eps) {
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; ++i) {
        std::size_t pivotRow = i;
        double pivotAbs = std::abs(a.get(i, i));

        for (std::size_t r = i + 1; r < n; ++r) {
            const double candidate = std::abs(a.get(r, i));
            if (candidate > pivotAbs) {
                pivotAbs = candidate;
                pivotRow = r;
            }
        }

        // Если в столбце подходящий элемент не найден, строку не
двигаем.
        // Ситуация потенциальной вырожденности будет дополнительно
проверена позже.
        if (pivotAbs <= eps) {
            continue;
        }

        if (pivotRow != i) {
            a.swapRows(i, pivotRow);
            std::swap(b[i], b[pivotRow]);
        }
    }
}

// Точная проверка типа решений через ранги rank(A) и rank([A|b]).
// Используется только для относительно небольших размеров, потому что
алгоритм O(n^3).
RankAnalysisResult analyzeByRanks(const SparseMatrix& a, const
std::vector<double>& b, double eps) {
    const std::size_t n = a.size();
    std::vector<std::vector<double>> aug(n, std::vector<double>(n + 1,
0.0));

    // Переносим разреженную матрицу в плотный вид для удобства гауссова
исключения.
    for (std::size_t i = 0; i < n; ++i) {
        for (const auto& [col, value] : a.row(i)) {
            aug[i][col] = value;
        }
        aug[i][n] = b[i];
    }

    std::size_t row = 0;
    for (std::size_t col = 0; col < n && row < n; ++col) {
        std::size_t pivot = row;
        double maxAbs = std::abs(aug[row][col]);
        for (std::size_t r = row + 1; r < n; ++r) {
            const double curAbs = std::abs(aug[r][col]);
            if (curAbs > maxAbs) {
                maxAbs = curAbs;
                pivot = r;
            }
        }
    }
}

```

```

    }
}

if (maxAbs <= eps) {
    continue;
}

if (pivot != row) {
    std::swap(aug[pivot], aug[row]);
}

for (std::size_t r = row + 1; r < n; ++r) {
    if (std::abs(aug[r][col]) <= eps) {
        continue;
    }
    const double factor = aug[r][col] / aug[row][col];
    for (std::size_t c = col; c <= n; ++c) {
        aug[r][c] -= factor * aug[row][c];
    }
}

++row;
}

std::size_t rankA = 0;
std::size_t rankAug = 0;
for (std::size_t i = 0; i < n; ++i) {
    bool nonZeroA = false;
    for (std::size_t j = 0; j < n; ++j) {
        if (std::abs(aug[i][j]) > eps) {
            nonZeroA = true;
            break;
        }
    }

    const bool nonZeroB = std::abs(aug[i][n]) > eps;
    if (nonZeroA) {
        ++rankA;
    }
    if (nonZeroA || nonZeroB) {
        ++rankAug;
    }
}

RankAnalysisResult result;
if (rankA < rankAug) {
    result.type = SolutionType::NONE;
    result.message = "Система несовместна: rank(A) < rank([A|b]).";
} else if (rankA == rankAug && rankA < n) {
    result.type = SolutionType::INFINITE;
    result.message = "Система совместна, но имеет бесконечно много
решений: rank(A)=rank([A|b])<n.";
} else if (rankA == n) {
    result.type = SolutionType::UNIQUE;
    result.message = "Система имеет единственное решение:
rank(A)=rank([A|b])=n.";
} else {
    result.type = SolutionType::UNDETERMINED;
}

```

```

        result.message = "Не удалось классифицировать систему по рангу.";
    }
    return result;
}

// Форматирование вектора с ограничением количества выводимых элементов.
std::string vectorToString(const std::vector<double>& v, std::size_t
maxElements = 8) {
    std::ostringstream oss;
    oss << "[";
    for (std::size_t i = 0; i < v.size() && i < maxElements; ++i) {
        if (i > 0) {
            oss << ", ";
        }
        oss << v[i];
    }
    if (v.size() > maxElements) {
        oss << ", ...";
    }
    oss << "]";
    return oss.str();
}

} // namespace

SparseMatrix::SparseMatrix(std::size_t n) : rows_(n) {}

std::size_t SparseMatrix::size() const {
    return rows_.size();
}

void SparseMatrix::set(std::size_t row, std::size_t col, double value) {
    if (row >= rows_.size() || col >= rows_.size()) {
        throw std::out_of_range("Индекс строки/столбца выходит за пределы
матрицы.");
    }

    auto& r = rows_[row];
    auto it = std::find_if(r.begin(), r.end(), [col](const auto& item) {
return item.first == col; });

    constexpr double storageEps = 1e-15;
    if (std::abs(value) <= storageEps) {
        if (it != r.end()) {
            r.erase(it);
        }
        return;
    }

    if (it != r.end()) {
        it->second = value;
    } else {
        r.emplace_back(col, value);
    }
}

double SparseMatrix::get(std::size_t row, std::size_t col) const {
    if (row >= rows_.size() || col >= rows_.size()) {

```

```

        throw std::out_of_range("Индекс строки/столбца выходит за пределы
матрицы.");
    }
    const auto& r = rows_[row];
    auto it = std::find_if(r.begin(), r.end(), [col](const auto& item) {
return item.first == col; });
    return (it == r.end()) ? 0.0 : it->second;
}

const std::vector<std::pair<std::size_t, double>>&
SparseMatrix::row(std::size_t row) const {
    if (row >= rows_.size()) {
        throw std::out_of_range("Индекс строки выходит за пределы
матрицы.");
    }
    return rows_[row];
}

void SparseMatrix::swapRows(std::size_t r1, std::size_t r2) {
    if (r1 >= rows_.size() || r2 >= rows_.size()) {
        throw std::out_of_range("Индекс строки выходит за пределы
матрицы.");
    }
    if (r1 == r2) {
        return;
    }
    std::swap(rows_[r1], rows_[r2]);
}

std::size_t SparseMatrix::memoryBytes() const {
    std::size_t bytes = sizeof(*this);
    bytes += rows_.capacity() * sizeof(std::vector<std::pair<std::size_t,
double>>);
    for (const auto& r : rows_) {
        bytes += r.capacity() * sizeof(std::pair<std::size_t, double>);
    }
    return bytes;
}

SolveResult solveByJacobi(SparseMatrix a, std::vector<double> b, const
JacobiConfig& config) {
    SolveResult result;
    const std::size_t n = a.size();

    if (n == 0) {
        result.message = "Пустая матрица: решать нечего.";
        return result;
    }

    if (b.size() != n) {
        result.message = "Размер вектора b не совпадает с размером
матрицы A.";
        return result;
    }

    // Применяем выбранную стратегию выбора ведущего элемента.
    if (config.pivotStrategy == PivotStrategy::PARTIAL_COLUMN_PIVOTING) {
        applyPartialColumnPivoting(a, b, config.zeroEps);
    }
}

```

```

    }

    // Быстрый контроль несовместности.
    if (hasInconsistentZeroRow(a, b, config.zeroEps)) {
        result.solutionType = SolutionType::NONE;
        result.message = "Найдена строка вида 0 ... 0 | b_i (b_i != 0):
решений нет.";
        return result;
    }

    const bool strictlyDominant = isStrictlyDiagonallyDominant(a,
config.zeroEps);

    // Если строгая диагональная доминантность есть, решение единственно
    гарантировано.
    // Иначе для небольших n делаем точный ранговый анализ.
    if (strictlyDominant) {
        result.solutionType = SolutionType::UNIQUE;
    } else if (n <= config.exactRankCheckLimit) {
        const RankAnalysisResult rankResult = analyzeByRanks(a, b,
config.zeroEps);
        result.solutionType = rankResult.type;
        if (rankResult.type == SolutionType::NONE || rankResult.type ==
SolutionType::INFINITE) {
            result.message = rankResult.message;
            return result;
        }
    } else {
        result.solutionType = SolutionType::UNDETERMINED;
    }

    // Начальное приближение  $x(0) = 0$ .
    std::vector<double> prevX(n, 0.0);
    std::vector<double> nextX(n, 0.0);

    // Оценка памяти на основные структуры данных, участвующие в расчёте.
    result.estimatedMemoryBytes =
        a.memoryBytes() +
        b.capacity() * sizeof(double) +
        prevX.capacity() * sizeof(double) +
        nextX.capacity() * sizeof(double);

    double previousDelta = std::numeric_limits<double>::infinity();

    for (std::size_t iter = 1; iter <= config.maxIterations; ++iter) {
        double maxDelta = 0.0;

        for (std::size_t i = 0; i < n; ++i) {
            double diag = 0.0;
            double sum = 0.0;

            // В формуле Якоби используем только значения из предыдущей
            // итерации prevX.
            //  $x_i(k+1) = (b_i - \sum_{j \neq i} (a_{ij} * x_j(k))) / a_{ii}$ 
            for (const auto& [col, value] : a.row(i)) {
                if (col == i) {
                    diag = value;
                } else {

```



```

        sum += value * prevX[col];
    }
}

// Проверка невырожденности "в процессе работы":
// если на диагонали ноль, формула Якоби для данной строки
неприменима.
if (std::abs(diag) <= config.zeroEps) {
    result.message =
        "На итерации обнаружен нулевой (или близкий к нулю)
диагональный элемент. "
        "Матрица вырождена или некорректно переставлена для
метода Якоби.";
    result.iterations = iter - 1;
    result.achievedDelta = maxDelta;
    return result;
}

const double newValue = (b[i] - sum) / diag;
if (!std::isfinite(newValue)) {
    result.message = "Численная ошибка: получено нечисловое
значение в итерациях.";
    result.iterations = iter - 1;
    result.achievedDelta = maxDelta;
    return result;
}

nextX[i] = newValue;
maxDelta = std::max(maxDelta, std::abs(nextX[i] - prevX[i]));
}

result.iterations = iter;
result.achievedDelta = maxDelta;

if (maxDelta < config.tolerance) {
    result.success = true;
    result.converged = true;
    result.x = nextX;

    if (result.solutionType == SolutionType::UNDETERMINED) {
        result.message =
            "Метод сошёлся, но для большого недиагонально-
доминантного случая "
            "тип решения не был строго классифицирован ранговой
проверкой.";
    } else {
        result.message = "Метод Якоби успешно сошёлся. x = " +
vectorToString(result.x);
    }
    return result;
}

// Простейший контроль на явное расхождение: если норма стала
огромной, прерываем.
if (iter > 10 && maxDelta > previousDelta * 1e6) {
    result.message =
        "Итерации расходятся (резкий рост нормы). Вероятно, для
данной матрицы "

```

```

        "метод Якоби не подходит без дополнительного
преобразования.";
        result.x = nextX;
        return result;
    }

    previousDelta = maxDelta;
    std::swap(prevX, nextX);
}

// До заданной точности не дошли, но частичный результат сохраним.
result.success = false;
result.converged = false;
result.x = prevX;
result.message =
    "Достигнут предел итераций без сходимости. Увеличьте
maxIterations, ослабьте tolerance "
    "или преобразуйте систему (например,
перестановками/масштабированием).";
return result;
}

SparseMatrix buildHilbertMatrix(std::size_t n) {
    SparseMatrix h(n);
    for (std::size_t i = 0; i < n; ++i) {
        for (std::size_t j = 0; j < n; ++j) {
            h.set(i, j, 1.0 / static_cast<double>(i + j + 1));
        }
    }
    return h;
}

SparseMatrix buildTridiagonalMatrix(std::size_t n, double diagValue,
double offDiagValue) {
    SparseMatrix m(n);
    for (std::size_t i = 0; i < n; ++i) {
        m.set(i, i, diagValue);
        if (i > 0) {
            m.set(i, i - 1, offDiagValue);
        }
        if (i + 1 < n) {
            m.set(i, i + 1, offDiagValue);
        }
    }
    return m;
}

```

Название файла: main.cpp

```

#include "jacobi_solver.h"

#include <iostream>
#include <string>
#include <vector>

namespace {

// Преобразование типа решения в читаемую строку.

```

```

std::string solutionTypeToString(SolutionType t) {
    switch (t) {
        case SolutionType::UNIQUE:
            return "Единственное решение";
        case SolutionType::INFINITE:
            return "Бесконечно много решений";
        case SolutionType::NONE:
            return "Решений нет";
        case SolutionType::UNDETERMINED:
            return "Не удалось строго классифицировать";
    }
    return "Неизвестный тип";
}

} // namespace

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    // Ниже простой интерфейс ввода:
    // 1) n
    // 2) матрица A (n x n)
    // 3) вектор b (n)
    // 4) стратегия выбора ведущего элемента: 0 или 1
    //
    // Пример:
    // 3
    // 10 1 1
    // 2 10 1
    // 2 2 10
    // 12 13 14
    // 1
    std::size_t n = 0;
    if (!(std::cin >> n)) {
        std::cerr << "Ошибка: не удалось прочитать размер матрицы n.\n";
        return 1;
    }

    SparseMatrix a(n);
    std::vector<double> b(n, 0.0);

    for (std::size_t i = 0; i < n; ++i) {
        for (std::size_t j = 0; j < n; ++j) {
            double value = 0.0;
            if (!(std::cin >> value)) {
                std::cerr << "Ошибка: не удалось прочитать элемент A[" <<
i << "][" << j << "].\n";
                return 1;
            }
            a.set(i, j, value);
        }
    }

    for (std::size_t i = 0; i < n; ++i) {
        if (!(std::cin >> b[i])) {
            std::cerr << "Ошибка: не удалось прочитать элемент b[" << i
<< "].\n";

```

```

        return 1;
    }
}

int pivotChoice = 1;
if (!(std::cin >> pivotChoice)) {
    std::cerr << "Ошибка: не удалось прочитать код стратегии ведущего
элемента.\n";
    return 1;
}

JacobiConfig cfg;
cfg.tolerance = 1e-10;
cfg.maxIterations = 200000;
cfg.zeroEps = 1e-12;
cfg.exactRankCheckLimit = 250;
cfg.pivotStrategy = (pivotChoice == 1)
    ? PivotStrategy::PARTIAL_COLUMN_PIVOTING
    : PivotStrategy::NONE;

const SolveResult res = solveByJacobi(a, b, cfg);

std::cout << "Статус: " << (res.success ? "Успех" : "Ошибка/Не
сошёлся") << "\n";
std::cout << "Сходимость: " << (res.converged ? "Да" : "Нет") <<
"\n";
std::cout << "Тип решения: " <<
solutionTypeToString(res.solutionType) << "\n";
std::cout << "Итераций: " << res.iterations << "\n";
std::cout << "Достигнутая норма ||x(k+1)-x(k)||inf: " <<
res.achievedDelta << "\n";
std::cout << "Оценка памяти (байт): " << res.estimatedMemoryBytes <<
"\n";
std::cout << "Сообщение: " << res.message << "\n";

if (res.success) {
    std::cout << "Решение x:\n";
    for (std::size_t i = 0; i < res.x.size(); ++i) {
        std::cout << "x[" << i << "] = " << res.x[i] << "\n";
    }
}

return 0;
}

```

Название файла: test.cpp

```

#include "jacobi_solver.h"

#include <chrono>
#include <cmath>
#include <fstream>
#include <functional>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>

```

```

#include <vector>

namespace {

// Удобная структура для описания одного теста.
struct TestCase {
    int id = 0;
    std::string name;
    SparseMatrix a;
    std::vector<double> b;
    std::vector<double> referenceX;
    JacobiConfig config;
    std::string inputDescription;

    // Проверка результата теста.
    // Возвращает true/false и пишет пояснение в outReason.
    std::function<bool(const SolveResult&, std::string& outReason)>
check;
};

// Умножение разреженной матрицы на вектор (для генерации эталонного b).
std::vector<double> multiply(const SparseMatrix& a, const
std::vector<double>& x) {
    const std::size_t n = a.size();
    std::vector<double> result(n, 0.0);
    for (std::size_t i = 0; i < n; ++i) {
        double sum = 0.0;
        for (const auto& [col, value] : a.row(i)) {
            sum += value * x[col];
        }
        result[i] = sum;
    }
    return result;
}

// Максимальная абсолютная ошибка между векторами.
double maxAbsError(const std::vector<double>& a, const
std::vector<double>& b) {
    if (a.size() != b.size()) {
        return std::numeric_limits<double>::infinity();
    }
    double mx = 0.0;
    for (std::size_t i = 0; i < a.size(); ++i) {
        mx = std::max(mx, std::abs(a[i] - b[i]));
    }
    return mx;
}

// Форматирование первых элементов вектора для отчёта.
std::string shortVector(const std::vector<double>& v, std::size_t
maxItems = 12) {
    std::ostringstream oss;
    oss << "[";
    for (std::size_t i = 0; i < v.size() && i < maxItems; ++i) {
        if (i > 0) {
            oss << ", ";
        }
        oss << std::setprecision(10) << v[i];
    }
}

```

```

    }
    if (v.size() > maxItems) {
        oss << ", ...";
    }
    oss << "]";
    return oss.str();
}

// Генератор циклической ленточной матрицы:
// A[i][i] = diagValue, а также несколько "смещённых" диагоналей по
модулю n.
// Такой тип матриц большой, разреженный и не сводится к простой
диагонали.
SparseMatrix buildCyclicBandedMatrix(
    std::size_t n,
    double diagValue,
    const std::vector<std::pair<int, double>>& offsetsAndValues) {
    SparseMatrix m(n);
    for (std::size_t i = 0; i < n; ++i) {
        m.set(i, i, diagValue);
        for (const auto& [offset, value] : offsetsAndValues) {
            const long long jSigned =
                (static_cast<long long>(i) + offset + static_cast<long
long>(n)) %
                static_cast<long long>(n);
            const std::size_t j = static_cast<std::size_t>(jSigned);
            m.set(i, j, value);
        }
    }
    return m;
}

// Генератор пятидиагональной матрицы (две соседние и две "вторые"
диагонали).
SparseMatrix buildPentadiagonalMatrix(
    std::size_t n,
    double diagValue,
    double firstOffDiag,
    double secondOffDiag) {
    SparseMatrix m(n);
    for (std::size_t i = 0; i < n; ++i) {
        m.set(i, i, diagValue);
        if (i >= 1) {
            m.set(i, i - 1, firstOffDiag);
        }
        if (i + 1 < n) {
            m.set(i, i + 1, firstOffDiag);
        }
        if (i >= 2) {
            m.set(i, i - 2, secondOffDiag);
        }
        if (i + 2 < n) {
            m.set(i, i + 2, secondOffDiag);
        }
    }
    return m;
}

```

```

// Генератор разреженной матрицы с "дальними" связями:
// для каждой строки добавляются связи с несколькими удалёнными
столбцами,
// что имитирует более сложную структуру, чем локальные диагонали.
SparseMatrix buildFarCoupledSparseMatrix(std::size_t n, double diagValue)
{
    SparseMatrix m(n);
    for (std::size_t i = 0; i < n; ++i) {
        // Набор детерминированных удалённых смещений.
        // Значения подобраны так, чтобы сохранялась диагональная
доминантность.
        const std::size_t j1 = (i + 17) % n;
        const std::size_t j2 = (i + 103) % n;
        const std::size_t j3 = (i + 509) % n;
        const std::size_t j4 = (i + n - 31) % n;
        const std::size_t j5 = (i + n - 211) % n;

        m.set(i, j1, -0.8);
        m.set(i, j2, 0.6);
        m.set(i, j3, -0.4);
        m.set(i, j4, 0.7);
        m.set(i, j5, -0.5);

        // Диагональ делаем достаточно большой для устойчивой сходимости
Якоби.
        m.set(i, i, diagValue);
    }
    return m;
}

// Создание базового конфигурационного шаблона метода Якоби.
JacobiConfig baseConfig() {
    JacobiConfig cfg;
    cfg.tolerance = 1e-10;
    cfg.maxIterations = 200000;
    cfg.zeroEps = 1e-12;
    cfg.exactRankCheckLimit = 250;
    cfg.pivotStrategy = PivotStrategy::PARTIAL_COLUMN_PIVOTING;
    return cfg;
}

} // namespace

int main() {
    std::vector<TestCase> tests;
    tests.reserve(10);

    // Тест 1: базовый случай с единственным решением.
    {
        SparseMatrix a(3);
        a.set(0, 0, 10.0); a.set(0, 1, 1.0); a.set(0, 2, 1.0);
        a.set(1, 0, 2.0); a.set(1, 1, 10.0); a.set(1, 2, 1.0);
        a.set(2, 0, 2.0); a.set(2, 1, 2.0); a.set(2, 2, 10.0);

        std::vector<double> xTrue = {1.0, 1.0, 1.0};
        std::vector<double> b = multiply(a, xTrue);
        JacobiConfig cfg = baseConfig();
        cfg.tolerance = 1e-12;
    }
}

```

```

tests.push_back(TestCase{
    1,
    "Базовый случай: единственное решение (3x3)",
    a,
    b,
    xTrue,
    cfg,
    "n=3, плотная диагонально-доминантная матрица,
x_true=[1,1,1]",
    [](const SolveResult& res, std::string& reason) {
        if (!res.success || !res.converged) {
            reason = "Ожидалась успешная сходимость, но её нет.";
            return false;
        }
        const double err = maxAbsError(res.x,
std::vector<double>{1.0, 1.0, 1.0});
        if (err > 1e-8) {
            reason = "Слишком большая ошибка решения: " +
std::to_string(err);
            return false;
        }
        reason = "Решение найдено, ошибка в пределах нормы.";
        return true;
    }
});

// Тест 2: система без решений (несовместная).
{
    SparseMatrix a(2);
    a.set(0, 0, 1.0); a.set(0, 1, 1.0);
    a.set(1, 0, 2.0); a.set(1, 1, 2.0);
    std::vector<double> b = {2.0, 5.0};
    JacobiConfig cfg = baseConfig();

    tests.push_back(TestCase{
        2,
        "Базовый случай: решений нет",
        a,
        b,
        {},
        cfg,
        "n=2, зависимые строки A, но несовместный b",
        [](const SolveResult& res, std::string& reason) {
            if (res.solutionType != SolutionType::NONE) {
                reason = "Ожидался тип решения NONE (нет решений).";
                return false;
            }
            reason = "Несовместность корректно обнаружена.";
            return true;
        }
    });
}

// Тест 3: бесконечно много решений.
{
    SparseMatrix a(2);

```



```

a.set(0, 0, 1.0); a.set(0, 1, 1.0);
a.set(1, 0, 2.0); a.set(1, 1, 2.0);
std::vector<double> b = {2.0, 4.0};
JacobiConfig cfg = baseConfig();

tests.push_back(TestCase{
    3,
    "Базовый случай: бесконечно много решений",
    a,
    b,
    {},
    cfg,
    "n=2, зависимые строки A и согласованный b",
    [](const SolveResult& res, std::string& reason) {
        if (res.solutionType != SolutionType::INFINITE) {
            reason = "Ожидался тип решения INFINITE.";
            return false;
        }
        reason = "Случай бесконечного множества решений корректно
обнаружен.";
        return true;
    }
});
}

// Тест 4: "абсурдно большая" матрица 1000x1000.
// Используется трёхдиагональная матрица, чтобы тест был реалистичен
по памяти.
{
    const std::size_t n = 1000;
    SparseMatrix a = buildTridiagonalMatrix(n, 10.0, -1.0);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE; // Для этой матрицы
перестановки не нужны.
    cfg.tolerance = 1e-8;
    cfg.maxIterations = 5000;

    tests.push_back(TestCase{
        4,
        "Большая матрица: 1000x1000 (трёхдиагональная)",
        a,
        b,
        xTrue,
        cfg,
        "n=1000, tridiagonal(diag=10, offdiag=-1), x_true=ones",
        [xTrue](const SolveResult& res, std::string& reason) {
            if (!res.success || !res.converged) {
                reason = "Для 1000x1000 ожидалась сходимость, но она
не достигнута.";
                return false;
            }
            const double err = maxAbsError(res.x, xTrue);
            if (err > 1e-5) {
                reason = "Слишком большая ошибка на 1000x1000: " +
std::to_string(err);
                return false;
            }
        }
    });
}

```

```

        }
        reason = "Сходимость и точность для 1000x1000
подтверждены.";
        return true;
    }
    });
}

// Тест 5: "абсурдно большая" матрица 10000x10000.
{
    const std::size_t n = 10000;
    SparseMatrix a = buildTridiagonalMatrix(n, 10.0, -1.0);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE;
    cfg.tolerance = 1e-6;
    cfg.maxIterations = 2000;
    cfg.exactRankCheckLimit = 150; // Явно отключаем дорогой ранговый
анализ для такого n.

    tests.push_back(TestCase{
        5,
        "Большая матрица: 10000x10000 (трёхдиагональная)",
        a,
        b,
        xTrue,
        cfg,
        "n=10000, tridiagonal(diag=10, offdiag=-1), x_true=ones",
        [xTrue](const SolveResult& res, std::string& reason) {
            if (!res.success || !res.converged) {
                reason = "Для 10000x10000 ожидалась сходимость, но её
нет.";
                return false;
            }
            const double err = maxAbsError(res.x, xTrue);
            if (err > 1e-4) {
                reason = "Слишком большая ошибка на 10000x10000: " +
std::to_string(err);
                return false;
            }
            reason = "Сходимость и точность для 10000x10000
подтверждены.";
            return true;
        }
    });
}

// Тест 6: матрица Гильберта (обычно сложна для Якоби, часто даёт
расходимость).
{
    const std::size_t n = 8;
    SparseMatrix a = buildHilbertMatrix(n);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.tolerance = 1e-12;
    cfg.maxIterations = 3000;

```

```

tests.push_back(TestCase{
    6,
    "Специальная матрица: Гильберта 8x8",
    a,
    b,
    xTrue,
    cfg,
    "n=8, Hilbert matrix, x_true=ones",
    [](const SolveResult& res, std::string& reason) {
        // Для классического Якоби матрица Гильберта, как
правило, неблагоприятна.
        // Здесь считаем корректным исходом отсутствие
сходимости.
        if (res.converged) {
            reason = "Неожиданно получена сходимость на Hilbert
8x8.";
            return false;
        }
        reason = "Ожидаемая трудность/расходимость на матрице
Гильберта подтверждена.";
        return true;
    }
});
}

// Тест 7: большая циклическая ленточная матрица 4000x4000.
{
    const std::size_t n = 4000;
    SparseMatrix a = buildCyclicBandedMatrix(
        n,
        10.0,
        {
            {1, -1.0}, {-1, -1.0},
            {17, 0.4}, {-17, 0.4},
            {101, -0.3}, {-101, -0.3}
        }
    );
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE;
    cfg.tolerance = 1e-8;
    cfg.maxIterations = 4000;
    cfg.exactRankCheckLimit = 120;

    tests.push_back(TestCase{
        7,
        "Большая матрица: 4000x4000 (циклическая ленточная, 7
диагоналей)",
        a,
        b,
        xTrue,
        cfg,
        "n=4000, cyclic banded offsets={+-1, +-17, +-101}, diag=10,
x_true=ones",
        [xTrue](const SolveResult& res, std::string& reason) {
            if (!res.success || !res.converged) {

```

```

        reason = "Ожидалась сходимость на циклической
ленточной матрице 4000x4000.";
        return false;
    }
    const double err = maxAbsError(res.x, xTrue);
    if (err > 2e-5) {
        reason = "Слишком большая ошибка на cyclic-banded
4000x4000: " + std::to_string(err);
        return false;
    }
    reason = "Циклическая ленточная матрица 4000x4000 успешно
пройдена.";
    return true;
}
});
}

// Тест 8: очень большая циклическая ленточная матрица 12000x12000.
{
    const std::size_t n = 12000;
    SparseMatrix a = buildCyclicBandedMatrix(
        n,
        11.0,
        {
            {1, -1.0}, {-1, -1.0},
            {37, 0.5}, {-37, 0.5}
        }
    );
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE;
    cfg.tolerance = 2e-6;
    cfg.maxIterations = 2500;
    cfg.exactRankCheckLimit = 80;

    tests.push_back(TestCase{
        8,
        "Большая матрица: 12000x12000 (циклическая ленточная, 5
диагоналей)",
        a,
        b,
        xTrue,
        cfg,
        "n=12000, cyclic banded offsets={+1, +-37}, diag=11,
x_true=ones",
        [xTrue](const SolveResult& res, std::string& reason) {
            if (!res.success || !res.converged) {
                reason = "Ожидалась сходимость на циклической
ленточной матрице 12000x12000.";
                return false;
            }
            const double err = maxAbsError(res.x, xTrue);
            if (err > 3e-4) {
                reason = "Слишком большая ошибка на cyclic-banded
12000x12000: " + std::to_string(err);
                return false;
            }
        }
    });
}

```

```

        reason = "Циклическая ленточная матрица 12000x12000
успешно пройдена.";
        return true;
    }
    });
}

// Тест 9: большая пятидиагональная матрица 6000x6000.
{
    const std::size_t n = 6000;
    SparseMatrix a = buildPentadiagonalMatrix(n, 12.0, -1.0, 0.7);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE;
    cfg.tolerance = 1e-7;
    cfg.maxIterations = 3500;
    cfg.exactRankCheckLimit = 100;

    tests.push_back(TestCase{
        9,
        "Большая матрица: 6000x6000 (пятидиагональная)",
        a,
        b,
        xTrue,
        cfg,
        "n=6000, pentadiagonal(diag=12, off1=-1, off2=0.7),
x_true=ones",
        [xTrue](const SolveResult& res, std::string& reason) {
            if (!res.success || !res.converged) {
                reason = "Ожидалась сходимость на пятидиагональной
матрице 6000x6000.";
                return false;
            }
            const double err = maxAbsError(res.x, xTrue);
            if (err > 1e-4) {
                reason = "Слишком большая ошибка на pentadiagonal
6000x6000: " + std::to_string(err);
                return false;
            }
            reason = "Пятидиагональная матрица 6000x6000 успешно
пройдена.";
            return true;
        }
    });
}

// Тест 10: большая разреженная матрица с дальними связями 9000x9000.
{
    const std::size_t n = 9000;
    SparseMatrix a = buildFarCoupledSparseMatrix(n, 14.0);
    std::vector<double> xTrue(n, 1.0);
    std::vector<double> b = multiply(a, xTrue);
    JacobiConfig cfg = baseConfig();
    cfg.pivotStrategy = PivotStrategy::NONE;
    cfg.tolerance = 2e-6;
    cfg.maxIterations = 2500;
    cfg.exactRankCheckLimit = 80;

```

```

        tests.push_back(TestCase{
            10,
            "Большая матрица: 9000x9000 (разреженная с дальними
связями)",
            a,
            b,
            xTrue,
            cfg,
            "n=9000, sparse far-coupled offsets={+17,+103,+509,-31,-211},
diag=14, x_true=ones",
            [xTrue](const SolveResult& res, std::string& reason) {
                if (!res.success || !res.converged) {
                    reason = "Ожидалась сходимость на far-coupled матрице
9000x9000.";
                    return false;
                }
                const double err = maxAbsError(res.x, xTrue);
                if (err > 4e-4) {
                    reason = "Слишком большая ошибка на far-coupled
9000x9000: " + std::to_string(err);
                    return false;
                }
                reason = "Разреженная матрица с дальними связями
9000x9000 успешно пройдена.";
                return true;
            }
        });
    }

    std::ofstream report("test_report.txt", std::ios::trunc);
    if (!report.is_open()) {
        std::cerr << "Не удалось открыть test_report.txt для записи.\n";
        return 1;
    }

    report << "Отчёт по тестированию метода Якоби\n";
    report << "=====\n\n";

    int passed = 0;
    for (const TestCase& test : tests) {
        const auto start = std::chrono::high_resolution_clock::now();
        const SolveResult res = solveByJacobi(test.a, test.b,
test.config);
        const auto finish = std::chrono::high_resolution_clock::now();
        const auto elapsedMs =
            std::chrono::duration_cast<std::chrono::milliseconds>(finish
- start).count();

        std::string reason;
        const bool ok = test.check(res, reason);
        if (ok) {
            ++passed;
        }

        std::cout << "Test " << test.id << " - " << test.name << ": " <<
(ok ? "OK" : "Fail") << "\n";
    }

```

```

report << "Тест #" << test.id << ": " << test.name << "\n";
report << "Состояние: " << (ok ? "OK" : "Fail") << "\n";
report << "Комментарий проверки: " << reason << "\n\n";

report << "Ввод:\n";
report << test.inputDescription << "\n";
report << "Параметры: tolerance=" << test.config.tolerance
    << ", maxIterations=" << test.config.maxIterations
    << ", pivotStrategy="
    << (test.config.pivotStrategy ==
PivotStrategy::PARTIAL_COLUMN_PIVOTING ? "PARTIAL" : "NONE")
    << ", zeroEps=" << test.config.zeroEps
    << ", exactRankCheckLimit=" <<
test.config.exactRankCheckLimit << "\n\n";

report << "Вывод:\n";
report << "success=" << (res.success ? "true" : "false")
    << ", converged=" << (res.converged ? "true" : "false")
    << ", iterations=" << res.iterations
    << ", achievedDelta=" << std::setprecision(16) <<
res.achievedDelta << "\n";
report << "estimatedMemoryBytes=" << res.estimatedMemoryBytes <<
"\n";
report << "message=" << res.message << "\n";
if (!res.x.empty()) {
    report << "x(first values)=" << shortVector(res.x, 10) <<
"\n";
}
report << "\n";

report << "Время (мс): " << elapsedMs << "\n";
report << "Память (байт): " << res.estimatedMemoryBytes << "\n";
report << "-----\n\n";
}

std::cout << "Итог: " << passed << "/" <<
static_cast<int>(tests.size()) << " тестов успешно.\n";
std::cout << "Подробный отчёт: test_report.txt\n";

return (passed == static_cast<int>(tests.size())) ? 0 : 1;
}

```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Таблица тестирования задачи:

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
1	Базовый уникальный случай: $n=3$, диагонально- доминантная матрица, $x_{\text{true}}=[1,1,1]$	Единственное решение, метод сошёлся, $x \approx [1,1,1]$, iterations=24	0.000 мс	312 Б
2	Несовместная система: $n=2$, зависимые строки A, но несогласованный b	Решений нет, $\text{rank}(A)$ $< \text{rank}([A b])$, x отсутствует	0.000 мс	0 Б
3	Бесконечно много решений: $n=2$, зависимые строки A и согласованный b	Бесконечно много решений, $\text{rank}(A)=\text{rank}([A b])<n$, x не единственен	0.000 мс	0 Б
4	Большая 1000x1000: трёхдиагональная (diag=10, offdiag=-1), $x_{\text{true}}=\text{ones}$	Единственное решение, сходимость достигнута, $x \approx [1,1,\dots,1]$	0.000 мс	95 992 Б

5	Большая 10000x10000: трёхдиагональная (diag=10, offdiag=-1), x_true=ones	Единственное решение, сходимость достигнута, $x \approx [1, 1, \dots, 1]$	1.000 мс	959 992 Б
6	Матрица Гильберта: n=8, $H(i,j)=1/(i+j+1)$, $b=H*[1, \dots, 1]$	Сходимость не достигнута (численная неустойчивость для Якоби), корректная диагностика остановки	0.000 мс	1 432 Б
7	Большая 4000x4000: циклическая ленточная (7 диагоналей), x_true=ones	Единственное решение, метод сошёлся, $x \approx [1, 1, \dots, 1]$	1.000 мс	640 024 Б
8	Большая 12000x12000: циклическая ленточная (5 диагоналей), x_true=ones	Единственное решение, метод сошёлся, $x \approx [1, 1, \dots, 1]$	2.000 мс	1 536 024 Б
9	Большая 6000x6000: пятидиагональная	Единственное решение, метод сошёлся, $x \approx [1, 1, \dots, 1]$	1.000 мс	767 928 Б

	(diag=12, off1=-1, off2=0.7)			
10	Большая 9000x9000: разреженная с дальними связями (+17,+103,+509,- 31,-211)	Единственное решение, метод сошёлся, $x \approx [1,1,\dots,1]$	1.000 мс	1 296 024 Б

Таблица 1 – Тестирование алгоритма.